

PHASE 6: Core API Development

Project Title: Academic Credential Verification and Alumni Tracking System

Student: Dusabe Marie Rose

Phase Duration: 28 February – 13 March 2026

Introduction

This phase focused on developing the core backend functionality of the Academic Credential Verification and Alumni Tracking System. The objective was to design and implement REST APIs that allow the system to manage academic data stored in the database.

Using the capabilities provided by Spring Boot, the backend application was structured into multiple layers including entities, repositories, services, and controllers. These components work together to enable the system to perform Create, Read, Update, and Delete operations on the stored data.

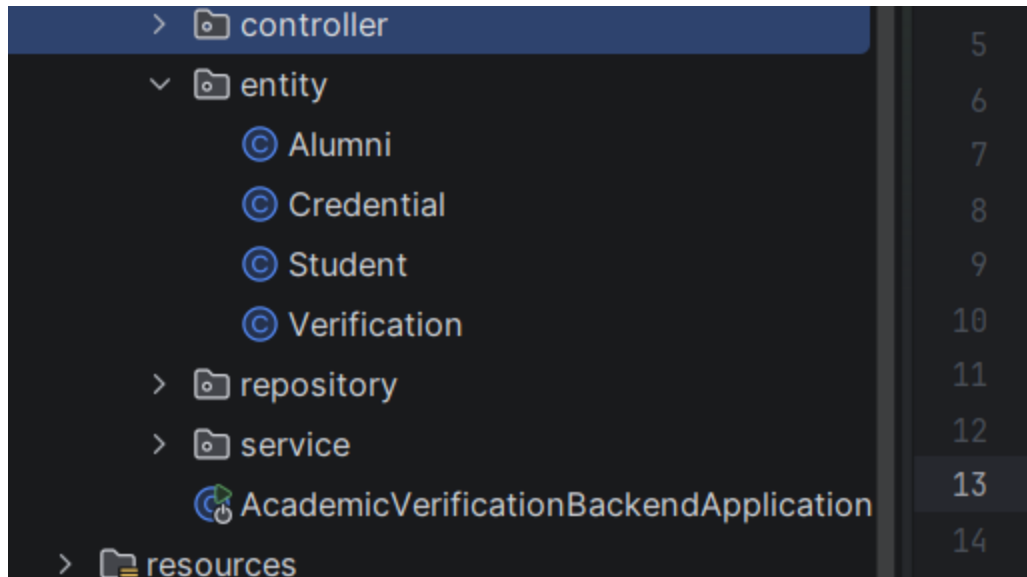
By completing this phase, the backend application became capable of receiving HTTP requests, processing them through the application logic, and interacting with the database.

Entity Class Development

The first step in this phase involved creating entity classes that represent database tables in the system. Entity classes define the structure of the data stored in the database and allow the application to map Java objects to relational database tables.

An entity class was created to represent alumni records. This class contains fields such as the alumni identifier, name, email, and graduation year. The entity was annotated with JPA annotations to allow the system to automatically generate and manage the corresponding database table.

This approach simplifies database management and allows developers to interact with data using Java objects instead of writing complex SQL queries.

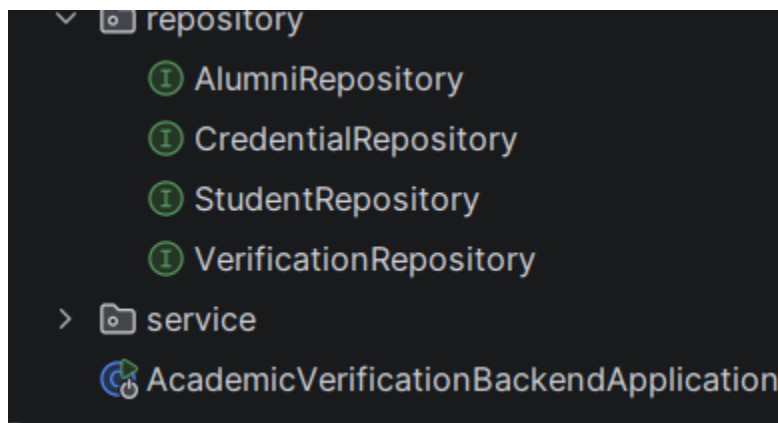


Repository Layer Implementation

After creating the entity class, the next step involved implementing the repository layer. The repository interface provides the functionality required to perform database operations such as saving records, retrieving records, updating records, and deleting records.

The repository interface extends the functionality provided by Spring Data JPA. This allows the system to automatically generate database queries without requiring manual SQL code.

Through this layer, the application can easily access and manipulate data stored in the database.

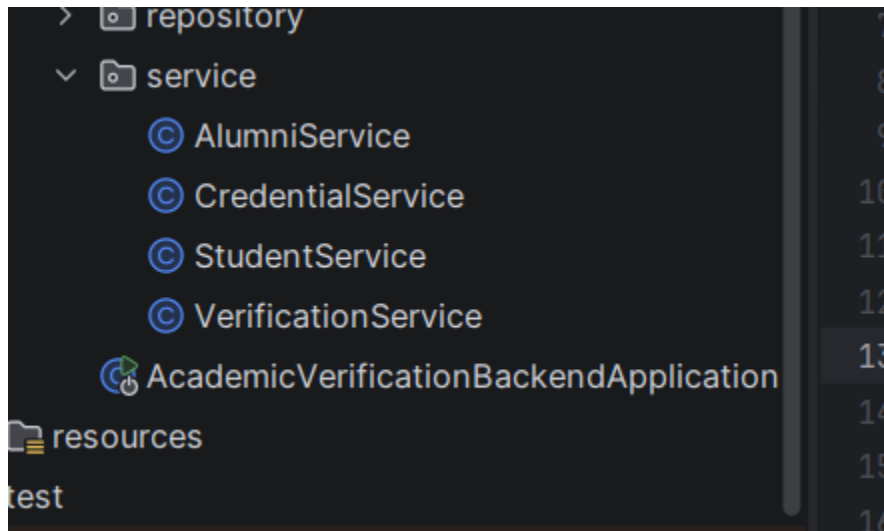


Service Layer Implementation

The service layer was implemented to handle the business logic of the application. This layer acts as an intermediary between the controller layer and the repository layer.

The service class contains methods responsible for retrieving alumni records, saving new records, updating existing records, and deleting records from the database.

Separating the business logic into the service layer improves the maintainability and scalability of the system by ensuring that controllers remain focused on handling HTTP requests.



REST Controller Development

The controller layer was developed to expose RESTful API endpoints that allow external applications or clients to interact with the backend system.

Using annotations provided by Spring Boot, several endpoints were created to manage alumni records. These endpoints allow the system to receive HTTP requests such as GET, POST, PUT, and DELETE and route them to the appropriate service methods.

The controller acts as the entry point for client requests and ensures that the correct operations are performed within the application.



CRUD Operations Implementation

The system was then configured to support full CRUD functionality. CRUD operations enable the application to create new records, retrieve existing records, update records, and delete records from the database.

Through the implemented API endpoints, users of the system can add new alumni information, retrieve stored data, update previously saved records, and remove records when necessary.

These operations form the core functionality of most data-driven applications and are essential for the proper functioning of the Academic Credential Verification system.

IP <http://localhost:8080/alumni>

POST <http://localhost:8080/alumni> Send

Params Auth Headers (8) Body Pre-req. Tests Settings

Raw JSON Body

```
1 {
2   "name": "Jean Claude",
3   "email": "jean@gmail.com",
4   "graduationYear": 2023
5 }
```

200 OK 450 ms 227 B Save Response

Pretty Raw Preview Visualize JSON

```
1 {
2   "alumni_id": 1,
3   "career_info": null,
4   "grad_year": 0,
5   "student": null
6 }
```

GET http://localhost:8080/alumni

HTTP http://localhost:8080/alumni Save

GET http://localhost:8080/alumni Send

Params Auth Headers (8) Body Pre-req. Tests Settings Cookies Beautify

raw JSON

```
1 [
2   "name": "Jean Claude",
3   "email": "jean@gmail.com",
4   "graduationYear": 2023
5 ]
```

Body 200 OK 860 ms 293 B Save Response

Pretty Raw Preview Visualize JSON

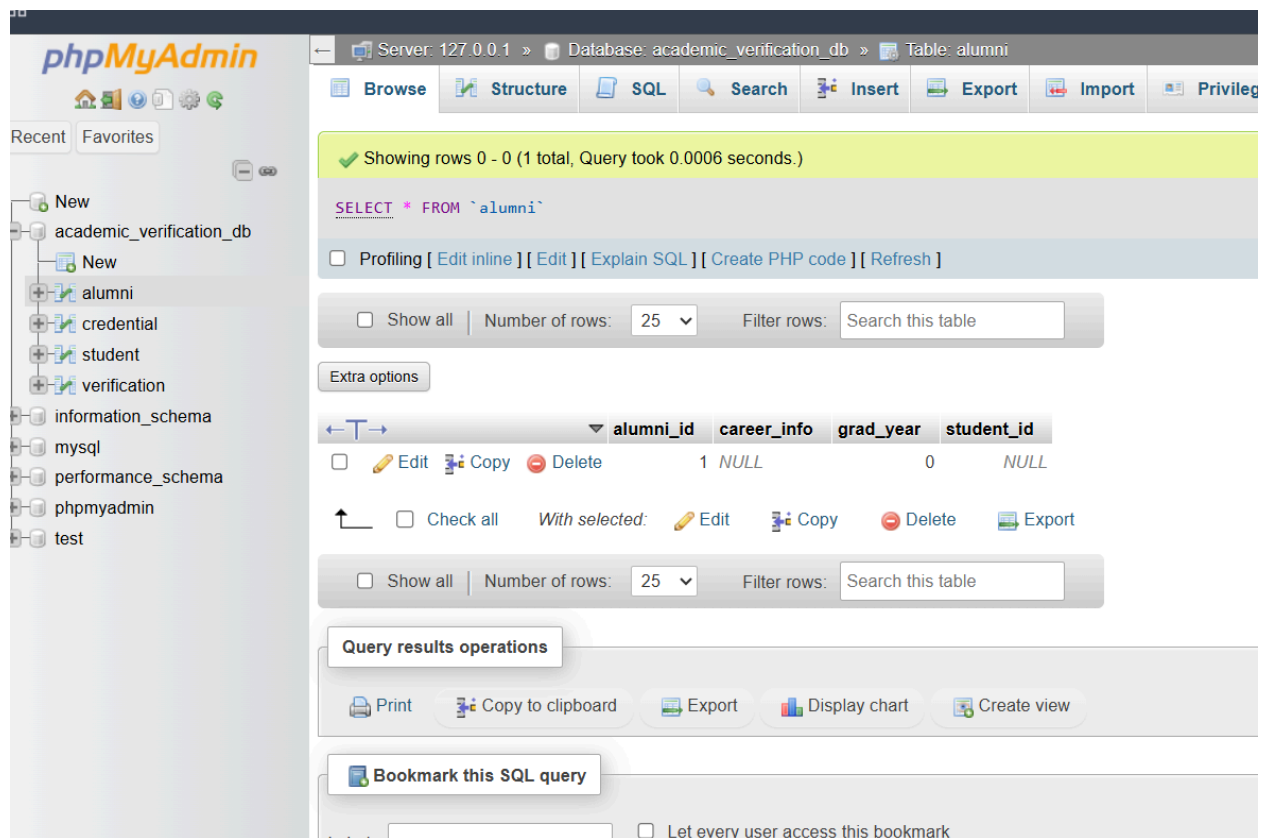
```
1 [
2   {
3     "alumni_id": 1,
4     "career_info": null,
5     "grad_year": 0,
6     "student": null
7   },
8   {
9     "alumni_id": 2,
10    "career_info": null,
11    "grad_year": 0,
12    "student": null
13  }
```

API Testing

To ensure that the developed APIs function correctly, the endpoints were tested using Postman. Each endpoint was tested by sending HTTP requests and verifying that the expected responses were returned.

Testing confirmed that the system could successfully communicate with the database and perform the intended operations.

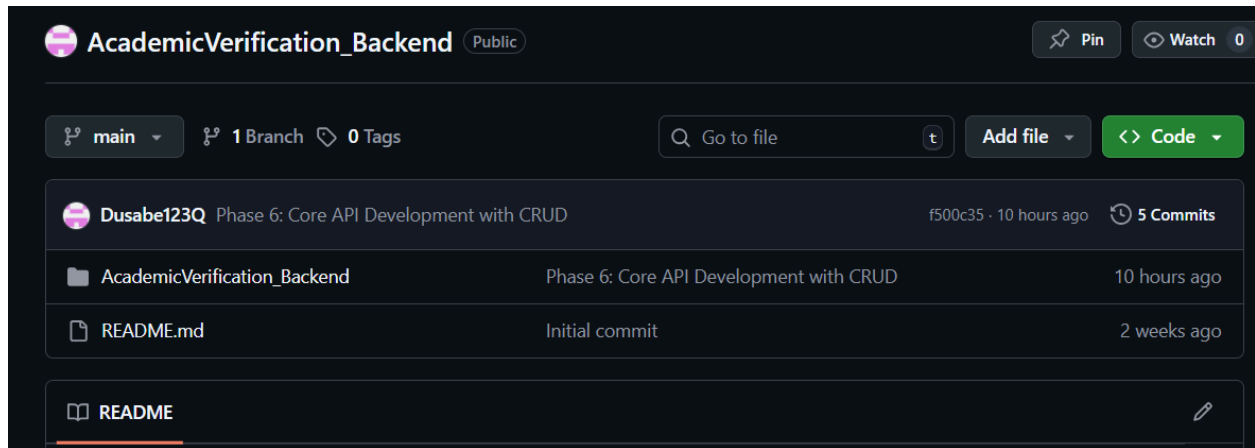
The API testing process verified that the backend application behaves correctly and that the CRUD operations are fully functional.



Publishing Updates to GitHub

After completing the API implementation and testing, the updated backend project was committed and pushed to the repository hosted on GitHub.

This step ensures that all development progress is recorded and accessible online. Version control also allows tracking of changes made during the development process.



Phase Deliverable

At the end of Phase 6, the backend application successfully provided functional REST APIs capable of performing full CRUD operations on alumni records stored in the database.

The system is now able to receive client requests, process them through the controller and service layers, interact with the database through the repository layer, and return appropriate responses.

These capabilities mark a significant milestone in the development of the Academic Credential Verification and Alumni Tracking System.